

Cinema Management System

Documento Técnico

Integrantes

- Ccoscco Alvis, Italo Frankdux
- Del Castillo Montoya, Christopher Brad
- García Valdivia, Ronald Pablo
- Ordóñez Ccoriccaza, Juan Carlos

1. Descripción del Sistema

CinemaManager es una aplicación web desarrollada con Python y Django que permite gestionar un sistema de cine completo. Incluye catálogo de películas, compra de entradas, comentarios con calificaciones y administración de funciones.

El sistema cuenta con dos roles:

- Cliente: puede explorar el catálogo, buscar películas, comprar entradas, comentar y guardar favoritos.
- Administrador: puede agregar, editar y eliminar películas, crear funciones con horarios y gestionar el stock de asientos.

Tecnologías utilizadas:

- Lenguaje: Python 3.10+
- Framework: Django
- Interfaz: HTML + CSS personalizado (Bootstrap-free)
- Base de datos: SQLite
- Testing: pytest con pytest-django

2. Validaciones del Sistema

El sistema implementa validaciones en tres niveles: validadores puros en `validators.py`, formularios Django (`forms.py`) y modelos (`models.py`), garantizando que ningún dato inválido llegue a la base de datos.

Módulo	Campo	Restricción	Aplica
Registro de usuario	Edad	$13 \leq \text{edad} \leq 100$	PE + AVL
Registro de usuario	Contraseña	Mínimo 8 caracteres	PE + AVL
Registro de usuario	Campos obligatorios	No vacíos (usuario, email)	PE
Compra de entradas	Cantidad de entradas	$1 \leq \text{entradas} \leq 10$	PE + AVL
Calificación de películas	Rating	$1 \leq \text{rating} \leq 5$	PE + AVL
Películas (admin)	Duración	$60 \leq \text{duración} \leq 300 \text{ min}$	PE + AVL
Películas (admin)	Año de estreno	$\text{año} \leq \text{año actual} (\leq 2026)$	PE + AVL
Comentarios	Longitud comentario	$1 \leq \text{chars} \leq 500$	PE + AVL
Funciones (admin)	Fecha y hora	fecha completa > momento actual	PE + AVL

Manejo de errores:

- Los formularios muestran mensajes amigables al usuario, sin stacktraces.
- Login requerido para comprar entradas y comentar — redirección automática al login.
- Tipos de dato validados en backend: se lanza `TypeError` para tipos incorrectos y `ValueError` para valores fuera de rango.
- El stock de asientos se verifica en tiempo real antes de confirmar una compra.
- No se pueden crear funciones con fecha pasada ni películas con año futuro.

3. Casos de Prueba — Partición de Equivalencia (PE)

Cada campo validado tiene al menos tres clases de equivalencia: válida, inválida por debajo del límite e inválida por encima. Se agrega una cuarta clase para tipos de dato incorrectos.

Campo	CE1 Válida	CE2 Inválida baja	CE3 Inválida alta	CE4 Tipo incorrecto
Edad	20, 50, 99	-1, 0, 12	101, 150, 999	"veinte", 20.5, None
Contraseña	"abcd1234"	"" , "abc", "1234567"	—	12345678, None

Entradas	1, 5, 10	-1, 0	11, 50, 999	"cinco", 5.5, None
Rating	1, 3, 5	-1, 0	6, 10, 100	"cinco", 3.5, None
Duración (min)	60, 120, 300	0, 30, 59	301, 400, 999	"noventa", 90.5, None
Año película	2026, 2000, 1950	—	2027, 2099	"2024", 2024.0, None
Comentario	"Buena película"	"", " "	501+ chars	123, None
Fecha función	ahora + 1 día	ahora - 1 día	—	"2027-01-01", None

4. Casos de Prueba — Análisis de Valores Límite (AVL)

Se prueba el comportamiento exactamente en la frontera, justo por debajo (−1) y justo por encima (+1) de cada límite definido.

Campo	Lím. inf.	inf−1	inf+1	Lím. sup.	sup−1	sup+1
Edad	13 ✓	12 ✗	14 ✓	100 ✓	99 ✓	101 ✗
Contraseña	8 ✓	7 ✗	9 ✓	—	—	—
Entradas	1 ✓	0 ✗	2 ✓	10 ✓	9 ✓	11 ✗
Rating	1 ✓	0 ✗	2 ✓	5 ✓	4 ✓	6 ✗
Duración (min)	60 ✓	59 ✗	61 ✓	300 ✓	299 ✓	301 ✗
Año película	—	—	—	2026 ✓	2025 ✓	2027 ✗
Comentario	1 ✓	0 ✗	2 ✓	500 ✓	499 ✓	501 ✗
Fecha función	+1 seg ✓	−1 seg ✗	+1 día ✓	—	—	—

Los tests están implementados en tests/test_validators.py usando pytest con @pytest.mark.parametrize. Total: 128 pruebas automatizadas, todas en verde.